

Truck Loading and Delivery Planning

CSE643 – Artificial Intelligence

Assignment 1: State-Space Search

1. Introduction

This project addresses the problem of planning the loading and delivery of packages at a logistics dispatch centre using state-space search.

Packages arrive at the dispatch centre over time and must be assigned to delivery trucks. Each package has an arrival time and a destination located along a common highway. Trucks have a fixed carrying capacity and can be reused after completing their delivery runs.

The objective is to find a delivery plan that provides a good trade-off between:

1. The number of physical trucks required simultaneously.
2. The average delay experienced by the packages.

The problem is formulated as a state-space search problem and solved using six search algorithms:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)
- Greedy Best-First Search
- A*
- Beam Search

The algorithms are evaluated using solution quality and search efficiency measures such as total cost, fleet size, average delivery delay, nodes expanded, nodes generated, and execution time.

2. Problem Formulation

2.1 Problem Description

The dispatch centre continuously receives packages. Each package is described by:

- A unique package ID
- An arrival time
- An integer destination

The destinations are assumed to lie along the same highway and therefore the travel time to a destination is proportional to its destination value.

Each truck has a fixed capacity and can carry multiple packages. Packages loaded onto a truck must satisfy the destination-ordering constraint used by the model.

The search algorithm must determine:

- When a truck should be dispatched.
- Which available packages should be loaded together.
- When it is beneficial to wait for additional packages.
- When a returning truck can be reused.
- How many trucks must be simultaneously active.

3. Objective Function

The original objective is to minimize:

$$C = \alpha F + \beta \frac{D}{N}$$

where:

- F = maximum number of simultaneously active trucks (fleet size)
- D = total delivery delay of all packages
- N = total number of packages
- α = weight associated with fleet size
- β = weight associated with delivery delay

For a package p , its delivery delay is:

$$Delay_p = DeliveryTime_p - ArrivalTime_p$$

Therefore:

$$D = \sum_{p=1}^N Delay_p$$

and:

$$AverageDelay = \frac{D}{N}$$

Scaled Search Objective

To avoid repeatedly using the division by N during search, the implementation uses the mathematically equivalent scaled objective:

$$C' = N\alpha F + \beta D$$

Since N is fixed for a given problem instance:

$$C' = NC$$

Therefore, minimizing C' produces exactly the same optimal solution as minimizing C .

4. State-Space Representation

A search state contains the information necessary to determine all future decisions and costs.

The state records information such as:

- Current simulation time
- Packages that have already been assigned to dispatched trucks
- Truck trip information
- Truck return times
- Current fleet size
- Path cost

The state representation is designed so that two states are considered identical only when they have the same relevant future behavior.

The accumulated path cost $g(n)$ is not used as part of state identity because the same logical state can be reached through different paths with different costs.

5. Actions

The search space contains two main types of actions.

5.1 Dispatch Action

A dispatch action selects a non-empty subset of currently available packages, subject to:

- Truck capacity
- Package arrival constraints
- Destination ordering constraints

The selected packages are assigned to a truck trip.

For a truck dispatched at time t with farthest destination d_{max} :

$$ReturnTime = t + 2d_{max}$$

For an individual package with destination d :

$$DeliveryTime = t + d$$

Therefore:

$$Delay = t + d - ArrivalTime$$

5.2 Wait Action

The search can choose to wait instead of immediately dispatching a truck.

Rather than generating arbitrary time values, the implementation advances time to the next relevant event:

- Arrival of a new package
- Return of an active truck

This keeps the state space finite and avoids exploring unnecessary intermediate time values.

Waiting is particularly important because trucks are reusable. Waiting for a returning truck can reduce the required fleet size at the expense of increased delivery delay.

6. Modeling Assumptions

The following assumptions are used to make the problem well-defined.

6.1 Truck Reuse

Trucks are treated as physical resources. After completing their delivery run, they return to the dispatch centre and can be reused.

Therefore, the same physical truck may perform multiple trips.

6.2 Fleet Size

Fleet size is defined as:

$$F = \max_t(\text{number of simultaneously active trucks})$$

It is therefore **not** the total number of truck dispatches.

For example, three sequential trips can have:

$$FleetSize = 1$$

if the same physical truck is reused for all three trips.

6.3 Deterministic Travel Time

Travel time is deterministic and proportional to destination distance.

For dispatch time t :

$$DeliveryTime = t + Destination$$

and:

$$ReturnTime = t + 2 \times Destination$$

where the maximum destination of the truck determines its return time.

6.4 Destination Ordering

Packages loaded onto a truck are required to appear in non-decreasing order of destination.

This reflects the assumption that packages are unloaded from the front of the truck and therefore packages for earlier destinations should not be blocked by packages for farther destinations.

6.5 Package Assignment

Once a package is assigned to a dispatched truck, its future delivery time is deterministic. Therefore, the package is treated as completed/assigned from the search perspective, while its actual delivery time is still used when calculating its delay.

6.6 Waiting

Waiting is allowed and is represented explicitly as a search action.

This allows the algorithm to trade additional package delay against reduced fleet requirements through truck reuse.

7. Search Algorithms

The same state-space model and successor function are used by all six algorithms.

7.1 Breadth-First Search

BFS expands states in increasing depth.

It is useful as an uninformed baseline but does not directly optimize the weighted delivery objective.

7.2 Depth-First Search

DFS explores one branch deeply before backtracking.

It can find solutions with very few expansions in some cases, but the first solution found is not necessarily the lowest-cost solution.

7.3 Uniform Cost Search

UCS expands the state with the smallest accumulated path cost:

$$g(n)$$

Since all transition costs are non-negative, UCS provides the optimal solution for the implemented state space.

UCS is therefore used as an important optimal-cost reference for evaluating the other algorithms.

7.4 Greedy Best-First Search

Greedy Best-First Search prioritizes states according to their heuristic value:

$$f(n) = h(n)$$

It can find solutions quickly but does not guarantee optimality.

7.5 A*

A* combines the accumulated cost and heuristic estimate:

$$f(n) = g(n) + h(n)$$

With an admissible and consistent heuristic, A* is guaranteed to return an optimal solution while potentially expanding fewer states than UCS.

7.6 Beam Search

Beam Search limits the number of states retained at each search level to a fixed beam width.

This reduces memory usage and search effort but sacrifices the optimality guarantee.

Different beam widths can therefore be used to study the trade-off between computational efficiency and solution quality.

8. Heuristics

Four heuristic functions were investigated.

8.1 h_1 — Remaining Package Count

$$h_1(s) = |\textit{RemainingPackages}|$$

This heuristic represents the number of packages still requiring assignment.

It is useful as a simple distance-to-goal estimate but is not expressed in the same units as the weighted objective and is therefore not used as the theoretically justified optimal A* heuristic.

8.2 h_2 — Fleet Growth Lower Bound

A naive fleet lower bound based on:

$$\left\lceil \frac{\textit{RemainingPackages}}{\textit{Capacity}} \right\rceil$$

is not necessarily valid for additional fleet cost because trucks can be reused.

A single truck may perform multiple trips.

Therefore, the implementation uses the conservative lower bound:

$$h_2(s) = 0$$

for guaranteed admissibility under the reusable-truck model.

8.3 h_3 — Minimum Remaining Delay

For each remaining package p , the earliest possible delivery delay is:

$$\max(\textit{CurrentTime}, \textit{ArrivalTime}_p) + \textit{Destination}_p - \textit{ArrivalTime}_p$$

Therefore:

$$h_3(s) = \beta \sum_{p \in \textit{Remaining}} (\max(\textit{CurrentTime}, \textit{ArrivalTime}_p) + \textit{Destination}_p - \textit{ArrivalTime}_p)$$

This represents a lower bound on the remaining delay component of the scaled objective.

8.4 h_4 — Combined Heuristic

The combined heuristic is:

$$h_4(s) = h_2(s) + h_3(s)$$

Since the current reusable-truck model gives:

$$h_2(s) = 0$$

the current implementation effectively has:

$$h_4(s) = h_3(s)$$

The heuristic has been empirically tested for admissibility and consistency on small reachable state spaces.
